Q2 – User-Defined Exceptions and Propagation in an Online Examination Portal

Hierarchy

```
Exception
└── ExamException (checked)
    ├── ExamClosedException            (checked: caller can show
"window closed")
    ├── NotRegisteredException         (checked: caller must handle)
    ├── FileTooLargeException          (checked)
    └── SubmissionStorageException     (checked: wraps IOException)
RuntimeException
└── PortalConfigException (unchecked: programming/config bug)
```

Why checked vs unchecked (Student A): Business-rule failures (closed window, not registered, file too large, storage failure) are expected and recoverable, so the compiler should force callers to handle them: checked. A configuration or programming mistake (e.g. null config) can't be recovered from by the caller: unchecked.

Code (three layers)

```
import java.io.IOException;

class ExamException extends Exception {
    ExamException(String m) { super(m); }
    ExamException(String m, Throwable c) { super(m, c); }
}
class ExamClosedException extends ExamException {
    ExamClosedException(String m) { super(m); }
}
class NotRegisteredException extends ExamException {
    NotRegisteredException(String m) { super(m); }
}
class FileTooLargeException extends ExamException {
    FileTooLargeException(String m) { super(m); }
}
class SubmissionStorageException extends ExamException {
    SubmissionStorageException(String m, Throwable c) { super(m, c); }
}

class Storage {
    void save(String file, long sizeKb) throws
SubmissionStorageException {
        try {
            if (sizeKb > 500) throw new IOException("Disk quota
exceeded");
            System.out.println("Saved " + file);
        } catch (IOException e) {
```

```
            throw new SubmissionStorageException("Could not store " +
file, e);
        }
    }
}

class SubmissionService {
    Storage storage = new Storage();
    void submit(String student, boolean registered, boolean open,
String file, long kb)
            throws ExamException {
        if (!open) throw new ExamClosedException("Exam window
closed");
        if (!registered) throw new NotRegisteredException(student + "
not registered");
        if (kb > 2000) throw new FileTooLargeException("File exceeds
limit: " + kb + "KB");
        storage.save(file, kb);                    // propagates
SubmissionStorageException
    }
}

public class Portal {                              // controller
    public static void main(String[] a) {
        SubmissionService svc = new SubmissionService();
        Object[][] tests = {
            {"S1", true, true, "a.pdf", 100L},
            {"S2", true, false, "b.pdf", 100L},
            {"S3", false, true, "c.pdf", 100L},
            {"S4", true, true, "d.pdf", 3000L},
            {"S5", true, true, "e.pdf", 800L}
        };
        for (Object[] t : tests) {
            try {
                svc.submit((String) t[0], (Boolean) t[1], (Boolean)
t[2], (String) t[3], (Long) t[4]);
            } catch (ExamClosedException | NotRegisteredException |
FileTooLargeException e) {
                System.out.println("Rejected: " + e.getMessage());
            } catch (SubmissionStorageException e) {
                System.out.println("Sorry, your file could not be
saved. Try again later.");
                System.out.println("  [log] cause = " + e.getCause());
// chaining evidence
            } catch (ExamException e) {
                System.out.println("Submission failed: " +
e.getMessage());
            }
        }
```

```
    }
}
```

Propagation path (Student A): `IOException` thrown in `Storage.save` → caught and wrapped into `SubmissionStorageException` → propagates through `SubmissionService.submit` (declared `throws ExamException`) → reaches `Portal.main` (controller) → handled in `catch (SubmissionStorageException)`. The student sees *"Sorry, your file could not be saved. Try again later."*, and the technical cause goes to the log.

Student B – Scenario / test table

Scenario	Input	Exception	Observed
Closed	open=false	`ExamClosedException`	"Rejected: Exam window closed"
Unregistered	registered=false	`NotRegisteredException`	`<fill>`
Too large	3000 KB	`FileTooLargeException`	`<fill>`
Storage failure	800 KB	`SubmissionStorageException` with cause `IOException`	`e.getCause().getMessage()` = "Disk quota exceeded"

Chaining: `e.getCause()` returns the `IOException`; `e.printStackTrace()` shows `Caused by: java.io.IOException: Disk quota exceeded` with its own frames. **If re-thrown without a cause** (`new SubmissionStorageException("Could not store x", null)` / using a constructor that drops it), `getCause()` returns `null`, the `Caused by:` section disappears, and the original type, message and line are lost, making diagnosis hard (and `e.getCause().getMessage()` would throw NPE).

Complete the Code – TODO 1

```
} catch (IOException e) {
    throw new SubmissionStorageException("Could not store " +
fileName, e);
}
```

Output:

```
Saved answer01.pdf
Error: Could not store answer02.pdf
Cause: Disk quota exceeded
```

If the IOException is not passed: `getCause()` returns `null`, so `e.getCause().getMessage()` throws a `NullPointerException` in main and the Cause line is never printed.

Joint conclusion

A `SubmissionStorageException` hides low-level details (disk, quota, paths) from the UI, keeps layers decoupled (the UI doesn't need to know storage uses files vs. a database), is meaningful to handle, and, with chaining, still keeps the original `IOException` for logs. A raw `IOException` leaks implementation details, is unclear to users, and couples the controller to the storage technology.

Alternative: unchecked exceptions

- Method signatures lose `throws ...` clauses (`save(...)`, `submit(...)`), and callers are no longer forced to use try/catch. Code is shorter, but a forgotten handler becomes a runtime crash instead of a compile error.
- **Time:** `Throwable` construction calls `fillInStackTrace()`, O(d) in stack depth d, for checked and unchecked alike. Making it unchecked changes nothing. The cost can be avoided by overriding `fillInStackTrace()` or using the protected constructor with `writableStackTrace=false`.
- **Space:** each exception holds a stack trace array, O(d). Chaining adds a reference to the cause, so one exception retains the whole chain: space = O(sum of the depths of all chained exceptions). A long chain keeps its causes (and their traces) alive until the outer one is garbage collected.